复习

1）JVM内存区域

程序计数器：线程私有的，唯一一个没有OOM的区域

本地方法栈：线程私有，调用native方法

虚拟机栈：线程私有，内部结构是栈帧结构（操作数栈、局部变量表、方法返回地址、动态链接），存编译期可知的8种基本数据类型和对象引用的

堆：线程共享的、分为老年代和新生代（新生代又分为Eden区和survivor区8:1:1）、存几乎所有的对象

方法区：线程共享的，别名叫永久代（在jdk1.8的时候叫元数据区），存类的描述信息、常量、符号引用、静态变量

目标

1）在JVM中是如何判断一个对象是否是垃圾

2）哪些对象可以作为GCRoot

3）一个对象被标记成垃圾之后一定是非死不可的吗？

4）有哪些垃圾收集算法

5）有哪些垃圾收集器，以及他们的特点

6）CMS的工作流程包括什么

7）理解吞吐量和最少停顿时间

1 如何判断一个对象是否是垃圾

1.1 引用计数法

Girl wentai = new Girl();//1个引用

Girl huihui = wentai;//new Girl这个对象就有两个引用指向了它

wentai = null;//此时new Girl()这个对象就只有一个引用指向他了

huihui=null;

当一个对象不存在任何引用指向它的时候，该对象就可以被当做是垃圾

存在一个缺陷：循环引用

1.2 可达性分析

GCRoot就是JVM根据算法自己进行选择的，GCRoot的选择思想就是要尽量选择生命周期长的对象

> 能作为GC Root:类加载器、Thread、虚拟机栈的本地变量表、static成员、常量引用、本地方法栈的变量等。

1.3 当一个对象被标记成垃圾的时候，是否非死不可呢

当一个人被判死刑了，都有可能会给他一次改过自新的机会，对象也会存在这种可能性

在java里面的条件是什么：

1）对象必须重写finalize方法

2）有且只有一次改过自新的机会（就是说如果该对象已经是第二次被判死刑，那即使满足第一个条件，我也要干掉你）

finalize这个方法一定被淘汰了，不允许在工作中用到该方法

在实际项目中虽然对象实现了finalize方法，但是如果不显示的调用system.gc的话，finalize方法不一定会被执行到

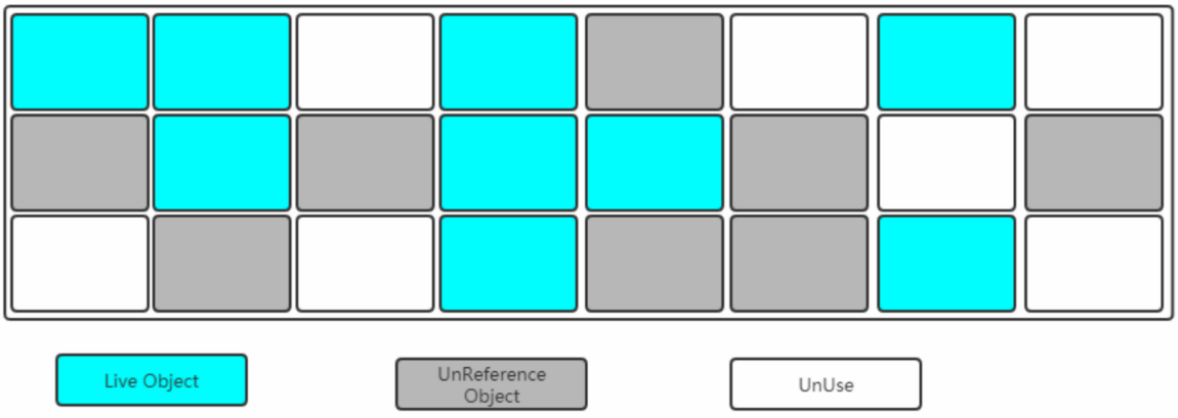
2 垃圾收集算法

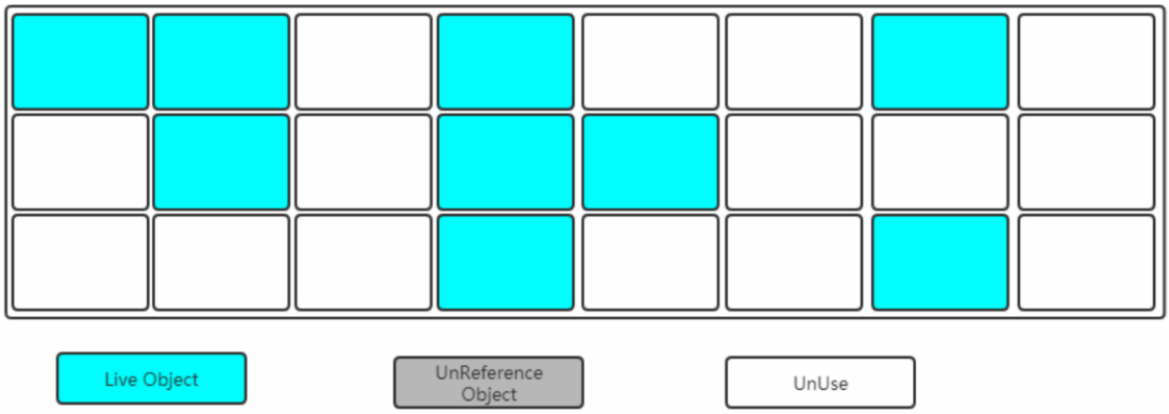
2.1 标记清除算法

标记-清除

用在老年代

就是通过GCRoot往下进行遍历查找





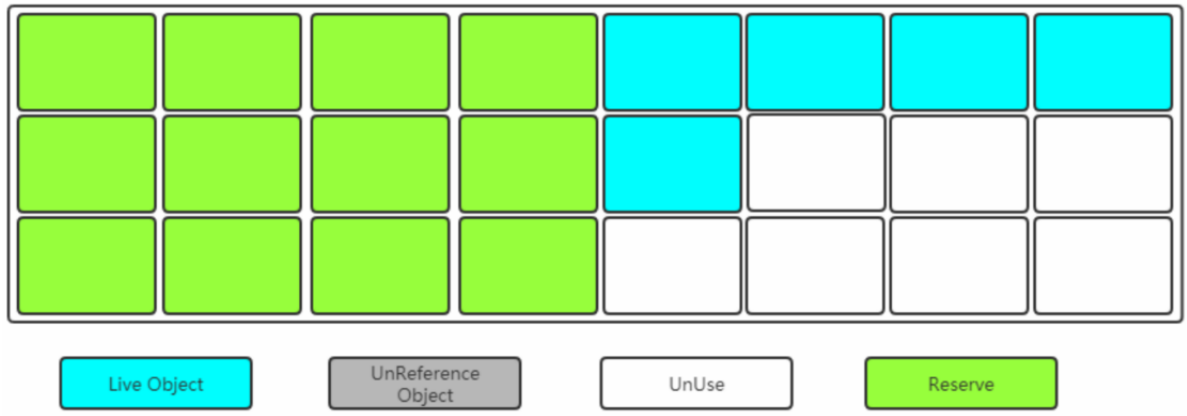
缺点：1）他会导致大量的空间碎片

2）标记和清除过程性能很低

2.2 复制算法

用在新生代

将新生代划分为Eden去和s0、s1区就是复制算法的思想体现

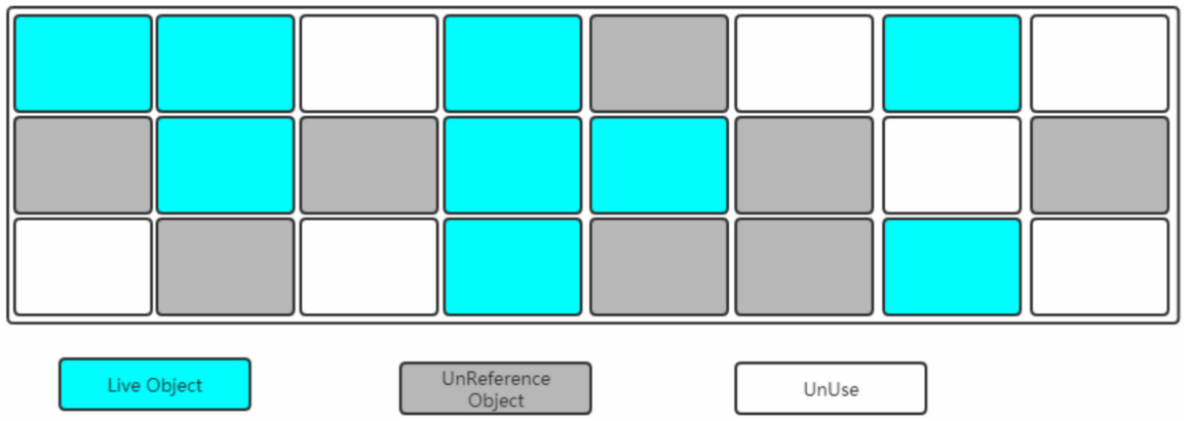


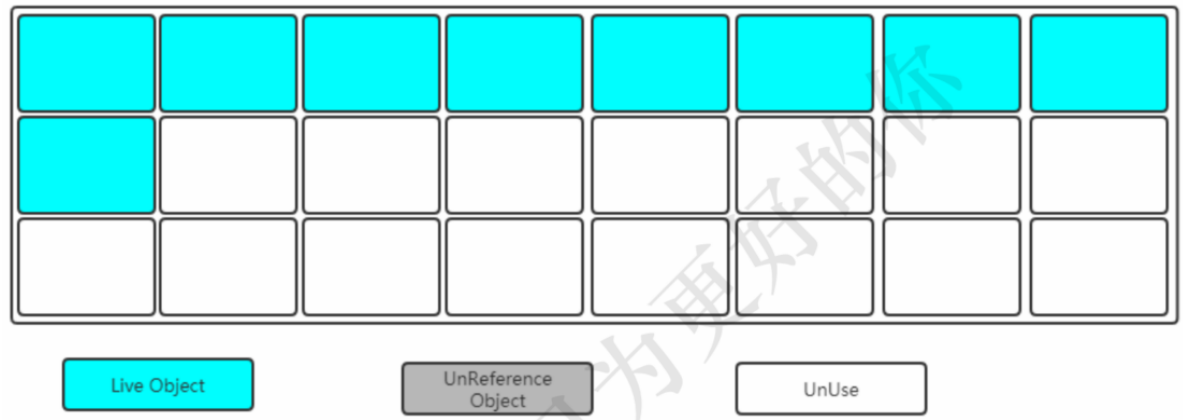
缺点：1）性能也不高

2）空间利用率降低了

2.3 标记整理算法

用在老年代





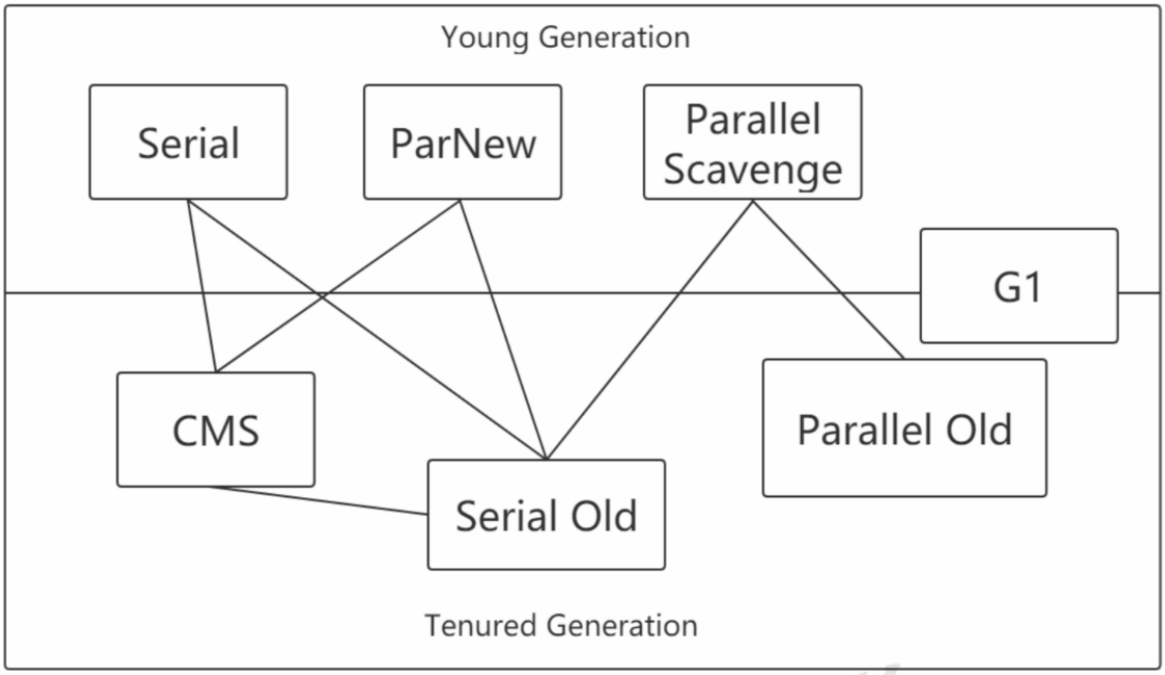
缺点：1）性能也不高

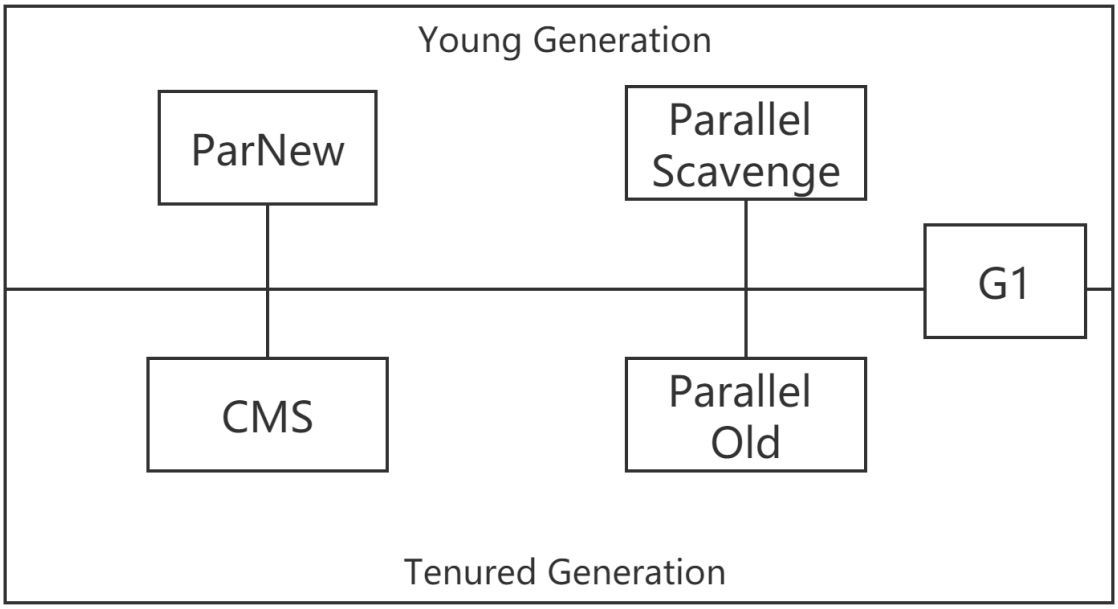
2.4 分代收集算法

就是将上面说的三种不同的垃圾收集算法运用在不同的区域当中，它不存在任何算法层面的创新

3 垃圾收集器

所有的垃圾收集器在工作过程中都会产生STW（Stop The World）

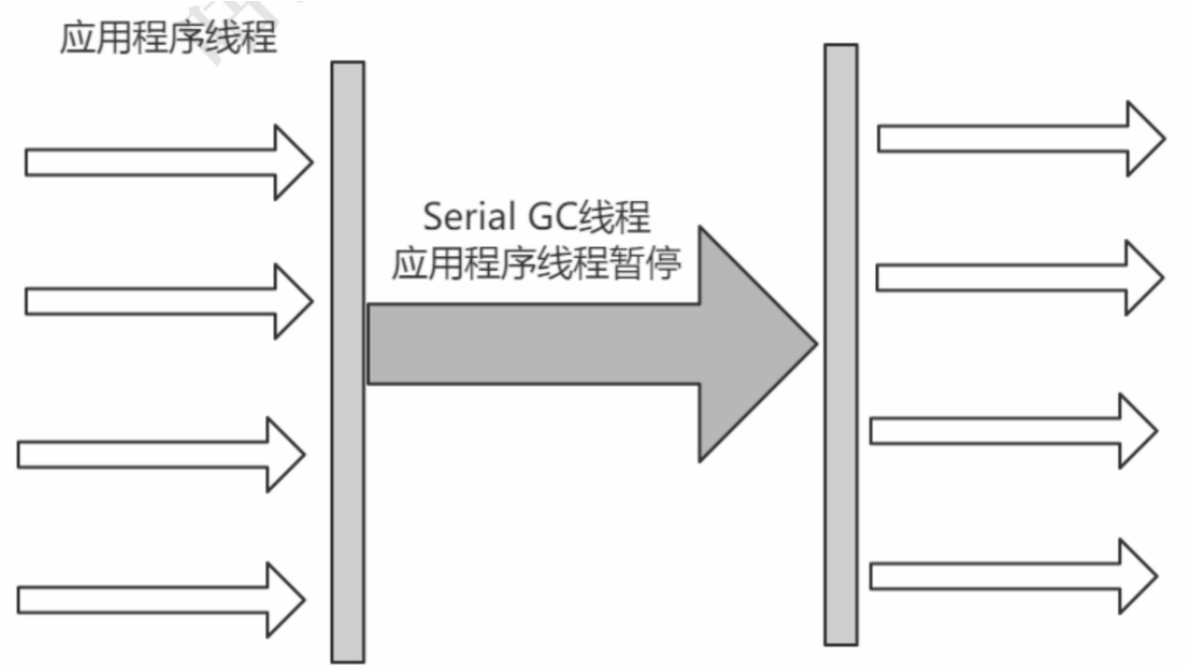




3.1 serial

1）复制算法

2）单线程的垃圾收集器

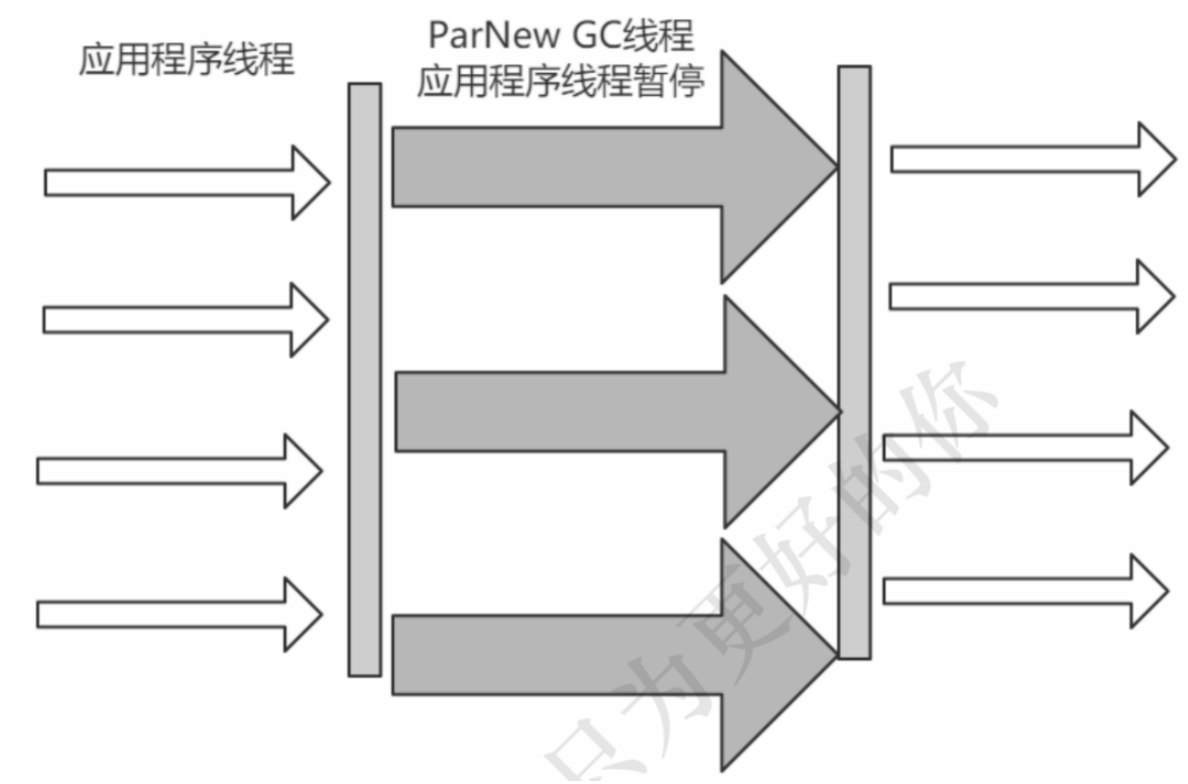


3.2 ParNew

对serial垃圾收集器进行优化了

1）复制算法

2）并行垃圾收集器（如果是单CPU的时候，ParNew的效率比serial低）



3.3 Parallel Scavenge

又是在ParNew的基础上进行优化

1）复制算法

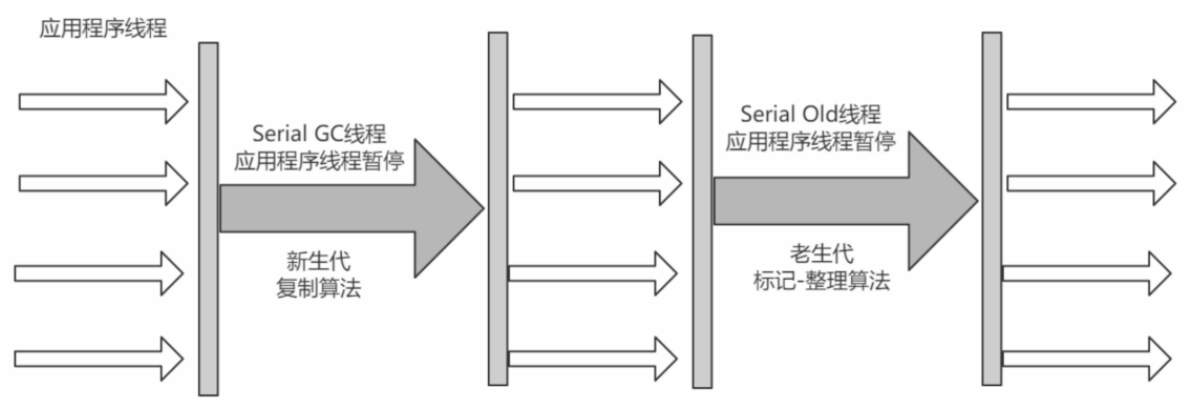
2）并行垃圾收集器

3）关注吞吐量（其实就是反映CPU的利用率的）

3.4 serial old

1）标记整理

2）单线程的



3.5 Parallel Old

1）标记整理

2）并行垃圾收集器

3）关注吞吐量

3.6 CMS

1）标记清除算法

2）并发垃圾收集器

3）关注的是最短停顿时间

4）工作步骤：分为4步（就是通过这4步进行了优化，大大的提高了性能）

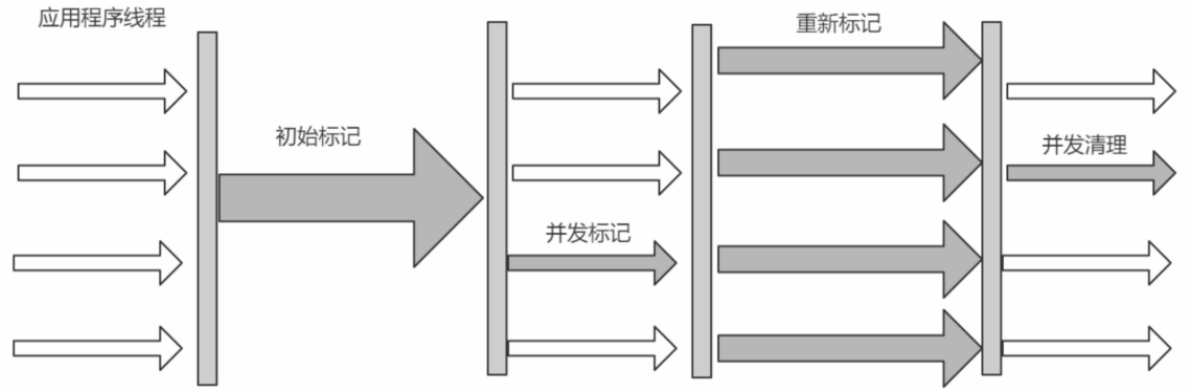
a.初始标记：只标记跟GCroot直接相连的对象（这个过程是STW的）

b.并发标记：从上一步的对象开始继续往下遍历，遍历所有对象，这个过程是跟用户线程并发执行的

在这个过程中会产生浮动垃圾。

c.重新标记：为了修正并发标记期间因用户程序继续运作而导致标记产生变动的那一部分对象的标记记录。这个阶段的停顿时间一般会比初始标记阶段稍长一些，但远比并发标记的时间要短（STW）

d.并发清除：就是跟用户线程一起执行，去回收垃圾对象



CMS有哪些优缺点：

优点：1、性能高 CPU的使用效率---->CPU在相同时间里，执行用户线程用的时间越多，性能就越高

2、停顿时间短

缺点：1、存在空间碎片

2、它会产生浮动垃圾

3、对CPU比较敏感。。。。如果在面试过程中碰到CPU突然飙升，怎么去排查

3.7 G1

1）分代收集算法

2）并发垃圾收集器

3）关注最短停顿时间

4）工作步骤：分为4步（就是通过这4步进行了优化，大大的提高了性能）

a.初始标记：只标记跟GCroot直接相连的对象（这个过程是STW的）

b.并发标记：从上一步的对象开始继续往下遍历，遍历所有对象，这个过程是跟用户线程并发执行的

在这个过程中会产生浮动垃圾。

c.最终标记：为了修正并发标记期间因用户程序继续运作而导致标记产生变动的那一部分对象的标记记录。这个阶段的停顿时间一般会比初始标记阶段稍长一些，但远比并发标记的时间要短（STW）

d.筛选回收：它会对垃圾对象进行评估，然后进行排序，再根据停顿时间去回收（STW）

1S

1k ---0.5s

10k -- 5s

5）可以通过参数去控制最短停顿时间

将最短停顿时间设置短一点，虽然停顿确实短了，但是堆内存是固定只有这么大的，每次垃圾回收都有一些对象回收不掉，随着时间推移，堆内存的空闲空间会越来越小，内存空间被占满的频率会越来越大，所以触发垃圾回收频率也会越来越大

6）G1垃圾收集器将堆内存划分为一个个的Region区

4 吞吐量和最短停顿时间

吞吐量=用户线程执行时间/（用户线程执行时间+垃圾回收线程执行时间）--->吞吐量越高，CPU效率越高

高吞吐量和短停顿时间是不能同时满足的